

IA - KR: PSC, cautare locala si algoritmi genetici

Teorie explicata + exemple tip examen rezolvate pas cu pas

Ce contine documentul:

- Probleme de satisfacere a constrangerilor (PSC/CSP): variabile, domenii, constrangeri, backtracking, forward checking.
- Exemple rezolvate: colorarea unei harti, N-regine, Sudoku ca PSC.
- Cautare locala si Hill Climbing: cum se alege vecinul, de ce apar minime/maxime locale, cum calculezi functia cost.
- Algoritmi genetici: codificare, fitness, selectie, crossover, mutatie, exemple numerice.
- Fisa rapida de memorat inainte de examen.

Contents

1	Harta subiectelor din cursurile 6 si 7	2
2	Probleme de satisfacere a constrangerilor - PSC/CSP	2
2.1	Definitie	2
2.2	Cautare standard vs PSC	2
2.3	Formularea unei PSC ca problema de cautare	3
3	Exemplu PSC: colorarea unei harti	3
3.1	Enunt tip examen	3
3.2	Graful de constrangeri	3
3.3	Rezolvare: formulare PSC	3
3.4	Forward checking pe exemplu	4
4	Backtracking si euristici pentru PSC	4
4.1	Backtracking simplu	4
4.2	MRV - Minimum Remaining Values	4
4.3	Degree heuristic	5
4.4	LCV - Least Constraining Value	5
5	Exemplu PSC: N-regine	5
5.1	Problema	5
5.2	Formulare 1 - variabile pentru fiecare patrat	5
5.3	Formulare 2 - o variabila pentru fiecare coloana	5
5.4	Exemplu pentru 4-regine	5
6	Exemplu PSC: Sudoku	6
6.1	Formulare	6
7	Cautare locala	6
7.1	Ideea principala	6
7.2	Comparatie cu cautarea clasica	6
8	Hill Climbing	7
8.1	Algoritmul pe pasi	7
8.2	Minimizare vs maximizare	7
8.3	Probleme posibile	7
9	Hill Climbing pentru 8-regine	7
9.1	Reprezentare	7
9.2	Vecinatate	8
9.3	Functie cost	8
9.4	Exemplu de calcul pentru 4-regine	8
10	Algoritmi genetici - idee generala	8
10.1	Motivatie	8
10.2	Structura unui algoritm genetic	9
10.3	Componente de memorat	9
10.4	Explorare vs exploatare	9
11	Codificarea solutiilor	9
11.1	Codificare binara	9
11.2	Exemplu: ONEMAX	9
11.3	Exemplu: rucsac	10
11.4	Codificare specifica pentru N-regine	10

12 Functia de fitness	10
12.1 Ce este fitnessul	10
12.2 Fitness pentru N-regine	10
12.3 Exemplu numeric	10
13 Selectia	11
13.1 Scopul selectiei	11
13.2 Selectie proportionala	11
13.3 Exemplu de examen	11
13.4 Problema valorilor negative	11
13.5 Selectie pe baza rangurilor	11
13.6 Selectie de tip turneu	12
13.7 Selectie prin trunchiere	12
14 Crossover / incrucisare	12
14.1 Scop	12
14.2 Crossover cu un punct	12
14.3 Crossover cu doua puncte	12
14.4 Crossover uniform	13
15 Mutatia	13
15.1 Scop	13
15.2 Mutatie binara	13
15.3 Probabilitate de mutatie	13
15.4 Mutatie pentru permutari	13
16 Exercitiu complet: algoritm genetic pentru 8-regine	14
16.1 Cerinta	14
17 Exercitiu complet: algoritm genetic pentru rucsac	14
17.1 Cerinta	14
18 Exercitiu complet: selectie proportionala	15
18.1 Cerinta	15
19 ExpectiMax - recapitulare utila	15
19.1 Cand folosim ExpectiMax	15
19.2 Reguli	15
19.3 Exemplu	15
20 Subiecte tip examen	16
21 Fisa rapida de memorat	16

1 Harta subiectelor din cursurile 6 si 7

Cele doua cursuri acopera partea de final din KR: dupa cautari si jocuri adversariale apar **ExpectiMax**, **PSC/CSP**, **cautare locala**, **hill climbing**, **calcul evolutiv** si **algoritmi genetici**. In Curs6 apar explicit: Alpha-Beta, ExpectiMax, probleme de satisfacere a constrangerilor, cautare locala si algoritmi genetici. In Curs7 se detaliaza calculul evolutiv si algoritmi genetici: codificare, fitness, selectie, crossover si mutatie.

Subiect	Ce trebuie sa stii	Ce poate pica la examen
PSC/CSP	variabile, domenii, constrangeri, asignare partiala, asignare completa, consistenta	formuleaza o problema ca PSC; colorare harta; Sudoku; N-regine; backtracking/forward checking
Cautare locala	stare curenta, vecinatate, functie cost/obiectiv, oprire la optim local	explica hill climbing; calculeaza h pentru 8-regine; arata de ce se blocheaza intr-un minim local
Algoritmi genetici	populatie, cromozom, fitness, selectie, crossover, mutatie	calculeaza probabilitati de selectie; aplica crossover; aplica mutatie; propune codificare si fitness pentru o problema
ExpectiMax	nod MAX si nod de sansa, medie ponderata	calculeaza valoarea unui nod de sansa; diferenta fata de Minimax

Important pentru examen: Daca ai putin timp, invata in ordinea: (1) PSC, (2) Hill Climbing, (3) Algoritmi genetici, (4) ExpectiMax recapitulare.

2 Probleme de satisfacere a constrangerilor - PSC/CSP

2.1 Definitie

O problema de satisfacere a constrangerilor este definita prin:

$$PSC = (X, D, C),$$

unde:

- $X = \{X_1, X_2, \dots, X_n\}$ este multimea variabilelor;
- $D = \{D_1, D_2, \dots, D_n\}$ este multimea domeniilor, unde D_i contine valorile posibile pentru X_i ;
- $C = \{C_1, C_2, \dots, C_m\}$ este multimea constrangerilor.

O **solutie** este o asignare completa a tuturor variabilelor, astfel incat toate constrangerile sa fie respectate.

Cum recunosti PSC la examen: vezi cuvinte precum variabile, domenii, culori, valori, asignari, constrangeri, Sudoku, N-regine, harta, criptaritmetica.

2.2 Cautare standard vs PSC

Cautare standard	PSC/CSP
Conteaza drumul pana la solutie.	Conteaza doar asignarea finala.
Actiunile pot avea costuri diferite.	De obicei fiecare pas asigneaza o variabila.
Adancimea solutiei poate varia.	Adancimea solutiei este de obicei numarul de variabile.

Exemple: labirint, 8-puzzle, drum minim. Exemple: colorare harta, Sudoku, N-regine.

2.3 Formularea unei PSC ca problema de cautare

Pentru o PSC, starea este o **asignare partiala**.

- Stare initiala: asignarea vida, $\{\}$.
- Succesor: aleg o variabila neasignata si ii dau o valoare din domeniu.
- Test scop: asignarea este completa si respecta toate constrangerile.

Exemplu:

$$\{WA = \text{rosu}, NT = \text{verde}\}$$

este o asignare partiala, nu o solutie completa.

3 Exemplu PSC: colorarea unei harti

3.1 Enunt tip examen

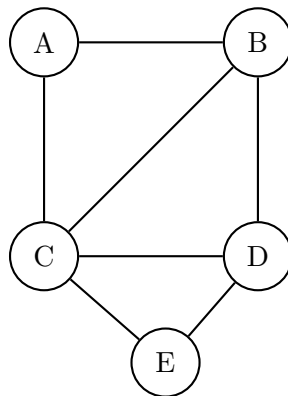
Avem regiunile A, B, C, D, E si trebuie colorate cu trei culori:

$$\{R, V, A\} = \{\text{rosu}, \text{verde}, \text{albastru}\}.$$

Regiunile vecine trebuie sa aiba culori diferite. Vecinatatile sunt:

$$A - B, A - C, B - C, B - D, C - D, C - E, D - E.$$

3.2 Graful de constrangeri



3.3 Rezolvare: formulare PSC

Rezolvare. Variabile:

$$X = \{A, B, C, D, E\}.$$

Domenii:

$$D_A = D_B = D_C = D_D = D_E = \{R, V, A\}.$$

Constrangeri:

$$A \neq B, A \neq C, B \neq C, B \neq D, C \neq D, C \neq E, D \neq E.$$

O solutie posibila este:

$$A = R, \quad B = V, \quad C = A, \quad D = R, \quad E = V.$$

Verificare:

- $A \neq B$: $R \neq V$ corect;
- $A \neq C$: $R \neq A$ corect;

- $B \neq C$: $V \neq A$ corect;
- $B \neq D$: $V \neq R$ corect;
- $C \neq D$: $A \neq R$ corect;
- $C \neq E$: $A \neq V$ corect;
- $D \neq E$: $R \neq V$ corect.

3.4 Forward checking pe exemplu

Ordinea variabilelor: A, B, C, D, E .

Initial:

$$D(A) = D(B) = D(C) = D(D) = D(E) = \{R, V, A\}.$$

Pasul 1: alegem $A = R$. Eliminam R din vecinii lui A , adica din B si C :

$$D(B) = \{V, A\}, \quad D(C) = \{V, A\}.$$

Pasul 2: alegem $B = V$. Eliminam V din vecinii neassignati ai lui B , adica din C si D :

$$D(C) = \{A\}, \quad D(D) = \{R, A\}.$$

Pasul 3: C este fortat:

$$C = A.$$

Eliminam A din vecinii neassignati ai lui C , adica din D si E :

$$D(D) = \{R\}, \quad D(E) = \{R, V\}.$$

Pasul 4: D este fortat:

$$D = R.$$

Eliminam R din E :

$$D(E) = \{V\}.$$

Pasul 5:

$$E = V.$$

Solutia finala:

$$A = R, \quad B = V, \quad C = A, \quad D = R, \quad E = V.$$

Important pentru examen: Forward checking nu gaseste mereu instant solutia, dar reduce domeniile in avans si detecteaza mai repede esecurile.

4 Backtracking si euristici pentru PSC

4.1 Backtracking simplu

Backtracking-ul construiesc o asignare partiala:

$$\{\} \rightarrow \{A = R\} \rightarrow \{A = R, B = V\} \rightarrow \dots$$

Daca apare conflict, revine la ultima decizie si incearca alta valoare.

4.2 MRV - Minimum Remaining Values

Alegi variabila cu cele mai putine valori ramase in domeniu. Ideea: alege variabila cea mai constransa.

Exemplu: dupa forward checking ai:

$$D(A) = \{R, V, A\}, \quad D(B) = \{V\}, \quad D(C) = \{R, A\}.$$

Alegi B , pentru ca are doar o valoare posibila.

4.3 Degree heuristic

Alegi variabila cu cei mai multi vecini neasignati. Ideea: variabila respectiva constrange puternic restul problemei.

4.4 LCV - Least Constraining Value

Alegi valoarea care elimina cele mai putine valori din domeniile vecinilor.

Formula de examen: MRV alege variabila cu cele mai putine valori ramase; degree heuristic rupe egalitatile alegand variabila cu cei mai multi vecini; LCV alege valoarea care lasa cele mai multe optiuni celorlalte variabile.

5 Exemplu PSC: N-regine

5.1 Problema

Trebuie sa asezam N regine pe o tabla $N \times N$, astfel incat nici doua regine sa nu se atace.

Doua regine se ataca daca sunt:

- pe aceeasi linie;
- pe aceeasi coloana;
- pe aceeasi diagonala.

5.2 Formulare 1 - variabile pentru fiecare patrat

Variabile:

$$X_{ij} \in \{0, 1\},$$

unde $X_{ij} = 1$ inseamna ca in patratul (i, j) este o regina.

Constrangeri:

- exact N regine in total;
- cel mult o regina pe fiecare linie;
- cel mult o regina pe fiecare coloana;
- cel mult o regina pe fiecare diagonala.

Dezavantaj: multe variabile si multe constrangeri.

5.3 Formulare 2 - o variabila pentru fiecare coloana

Variabile:

$$Q_i \in \{1, 2, \dots, N\},$$

unde Q_i este linia pe care se afla regina din coloana i .

Avantaj: avem automat exact o regina pe fiecare coloana.

Constrangeri pentru orice $i \neq j$:

$$Q_i \neq Q_j \quad (\text{nu aceeasi linie}),$$

$$|Q_i - Q_j| \neq |i - j| \quad (\text{nu aceeasi diagonala}).$$

5.4 Exemplu pentru 4-regine

Fie asignarea:

$$(Q_1, Q_2, Q_3, Q_4) = (2, 4, 1, 3).$$

Verificam liniile: 2, 4, 1, 3 sunt distincte, deci nu exista doua regine pe aceeasi linie.

Verificam diagonalele:

- $Q_1 = 2, Q_2 = 4$: $|2 - 4| = 2, |1 - 2| = 1$, nu sunt pe diagonala.

- $Q_1 = 2, Q_3 = 1$: $|2 - 1| = 1, |1 - 3| = 2$, nu sunt pe diagonala.
- $Q_1 = 2, Q_4 = 3$: $|2 - 3| = 1, |1 - 4| = 3$, nu sunt pe diagonala.
- $Q_2 = 4, Q_3 = 1$: $|4 - 1| = 3, |2 - 3| = 1$, nu sunt pe diagonala.
- $Q_2 = 4, Q_4 = 3$: $|4 - 3| = 1, |2 - 4| = 2$, nu sunt pe diagonala.
- $Q_3 = 1, Q_4 = 3$: $|1 - 3| = 2, |3 - 4| = 1$, nu sunt pe diagonala.

Deci este solutie.

6 Exemplu PSC: Sudoku

6.1 Formulare

Pentru Sudoku clasic 9×9 :

Variabile:

$$X_{ij}, \quad i, j \in \{1, \dots, 9\},$$

unde X_{ij} este valoarea din celula de pe linia i , coloana j .

Domenii:

$$D_{ij} = \{1, 2, \dots, 9\},$$

cu exceptia celulelor deja completate, care au domeniu singleton.

Constrangeri:

- pe fiecare linie, toate cele 9 valori sunt distincte;
- pe fiecare coloana, toate cele 9 valori sunt distincte;
- in fiecare patrat 3×3 , toate cele 9 valori sunt distincte.

Important pentru examen: La examen nu trebuie sa rezolvi Sudoku complet, ci sa stii sa il formulezi ca PSC.

7 Cautare locala

7.1 Ideea principala

Cautarea locala se foloseste cand nu conteaza drumul catre solutie, ci doar starea finala. In loc sa pastrezi o frontiera mare de noduri, pastrezi o singura stare curenta si o imbunatatesti.

- Stare curenta: solutia candidata actuala.
- Vecini: stari obtinute prin mici modificari locale.
- Functie cost: vrem sa o minimizam.
- Functie obiectiv: vrem sa o maximizam.

7.2 Comparatie cu cautarea clasica

Cautare clasica	Cautare locala
Pastreaza frontiera.	Nu are frontiera.
Cauta un drum de la start la scop.	Imbunatateste o stare curenta.
Poate fi completa si optima, in functie de algoritm.	In general incompleta si suboptimala.
Consuma multa memorie.	Consuma foarte putina memorie.
Exemple: BFS, UCS, A*.	Exemple: Hill Climbing, Simulated Annealing, algoritmi genetici.

8 Hill Climbing

8.1 Algoritmul pe pasi

Hill climbing este o cautare locala greedy.

Pasi Hill Climbing:

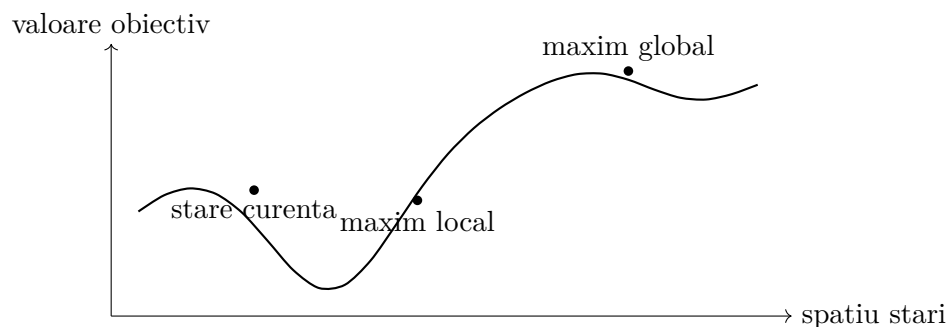
1. Pornesc dintr-o stare initiala, de obicei aleatoare.
2. Generez vecinii starii curente.
3. Calculez functia cost/obiectiv pentru fiecare vecin.
4. Aleg cel mai bun vecin.
5. Daca vecinul este mai bun decat starea curenta, ma mut in el.
6. Daca niciun vecin nu este mai bun, ma opresc.

8.2 Minimizare vs maximizare

Tip functie	Ce face hill climbing
Functie cost h	alege vecinul cu cost mai mic; vrea minim global
Functie obiectiv g	alege vecinul cu valoare mai mare; vrea maxim global

8.3 Probleme posibile

- **Minim/maxim local:** nu exista vecin mai bun, dar starea nu este solutie globala.
- **Platou:** multi vecini au aceeasi valoare, algoritmul nu stie clar in ce directie sa mearga.
- **Creasta:** directia buna necesita mai multi pasi care initial nu par buni.



9 Hill Climbing pentru 8-regine

9.1 Reprezentare

O stare poate fi reprezentata ca un vector:

$$(s_1, s_2, \dots, s_8),$$

unde s_i este linia pe care se afla regina din coloana i .

Exemplu:

$$(3, 2, 7, 5, 2, 4, 1, 1).$$

Aceasta inseamna ca:

- in coloana 1, regina este pe linia 3;
- in coloana 2, regina este pe linia 2;
- etc.

9.2 Vecinatate

Un vecin se obtine mutand o regina in alta linie, in aceeasi coloana.

Pentru 8 regine:

- avem 8 coloane;
- in fiecare coloana, regina poate fi mutata in 7 alte linii;

deci numarul de vecini este:

$$8 \cdot 7 = 56.$$

9.3 Functie cost

Functia cost:

$$h(s) = \# \text{perechi de regine care se ataca.}$$

Vrem:

$$\min h(s).$$

Solutia are:

$$h(s) = 0.$$

9.4 Exemplu de calcul pentru 4-regine

Luam starea:

$$s = (1, 1, 3, 4).$$

Reginele sunt in pozitiile:

$$(1, 1), (2, 1), (3, 3), (4, 4),$$

unde prima coordonata este coloana, a doua linia.

Numaram perechile care se ataca:

- $(1, 1)$ si $(2, 1)$: aceeasi linie, se ataca;
- $(3, 3)$ si $(4, 4)$: aceeasi diagonala, se ataca;
- $(1, 1)$ si $(3, 3)$: aceeasi diagonala, se ataca;
- $(1, 1)$ si $(4, 4)$: aceeasi diagonala, se ataca;
- $(2, 1)$ si $(3, 3)$: nu;
- $(2, 1)$ si $(4, 4)$: nu.

Deci:

$$h(s) = 4.$$

Un posibil vecin este:

$$s' = (2, 1, 3, 4),$$

adica mutam regina din coloana 1 de pe linia 1 pe linia 2. Calculam din nou $h(s')$ si alegem vecinul cu valoarea cea mai mica.

Important pentru examen: In problemele de examen, nu trebuie sa testezi toti cei 56 de vecini daca nu se cere explicit. Trebuie sa stii cum se definesc vecinii si cum se calculeaza functia cost.

10 Algoritmi genetici - idee generala

10.1 Motivatie

Algoritmii genetici se folosesc cand:

- spatiul solutiilor este foarte mare;
- functia cost/obiectiv are multe minime sau maxime locale;
- nu avem gradient sau nu vrem sa il calculam;
- metodele greedy sau hill climbing se blocheaza usor.

Ideea este sa nu tinem o singura stare curenta, ci o **populatie** de solutii candidate.

10.2 Structura unui algoritm genetic

Pasi algoritm genetic:

1. Initializez populatia aleator.
2. Calculez fitnessul fiecarui individ.
3. Selectez parinti, preferand indivizii cu fitness mare.
4. Generez urmasi prin crossover.
5. Aplic mutatie pe urmasi.
6. Calculez fitnessul urmasilor.
7. Selectez supravietuitorii pentru generatia urmatoare.
8. Repet pana la conditia de oprire.

10.3 Componente de memorat

Componenta	Rol
Populatie	multime de indivizi/solutii candidate
Cromozom	reprezentarea unei solutii
Gena	o componenta a cromozomului
Fitness	masoara calitatea unei solutii
Selectie	alege indivizi buni pentru reproducere sau supravietuire
Crossover	combina informatia din doi parinti
Mutatie	introduce variatie noua
Elitism	pastreaza cei mai buni indivizi neschimbati

10.4 Explorare vs exploatare

- **Explorare:** cautam regiuni noi din spatiul solutiilor. Este produsa de crossover si mutatie.
- **Exploatare:** folosim solutiile bune deja gasite. Este produsa de selectie.

11 Codificarea solutiilor

11.1 Codificare binara

Cromozomul este un sir de biti:

$$x = (x_1, x_2, \dots, x_n), \quad x_i \in \{0, 1\}.$$

Spatiul de cautare este:

$$\{0, 1\}^n.$$

11.2 Exemplu: ONEMAX

Problema ONEMAX cere maximizarea:

$$f(x) = x_1 + x_2 + \dots + x_n.$$

Pentru:

$$x = 101101,$$

fitnessul este:

$$f(x) = 1 + 0 + 1 + 1 + 0 + 1 = 4.$$

Solutia optima este:

$$111111,$$

cu fitness 6.

11.3 Exemplu: rucsac

Avem n obiecte. Cromozomul:

$$(s_1, s_2, \dots, s_n),$$

unde:

$$s_i = \begin{cases} 1, & \text{obiectul } i \text{ este selectat,} \\ 0, & \text{obiectul } i \text{ nu este selectat.} \end{cases}$$

Daca depasim capacitatea rucsacului, fitnessul trebuie penalizat.

11.4 Codificare specifica pentru N-regine

Pentru N-regine, o codificare buna este o permutare:

$$(s_1, s_2, \dots, s_N),$$

unde s_i este linia reginei din coloana i .

Avantaj: daca folosim permutare, liniile sunt distincte automat. Deci eliminam din start conflictele pe linii si coloane; raman de verificat diagonalele.

12 Functia de fitness

12.1 Ce este fitnessul

Fitnessul masoara cat de buna este o solutie. De obicei:

- fitness mare = solutie buna;
- fitness mic = solutie slaba.

12.2 Fitness pentru N-regine

Pentru N regine, numarul total de perechi de regine este:

$$\binom{N}{2} = \frac{N(N-1)}{2}.$$

Pentru $N = 8$:

$$\binom{8}{2} = 28.$$

Daca $a(s)$ este numarul de perechi care se ataca, o functie de fitness poate fi:

$$f(s) = 28 - a(s).$$

12.3 Exemplu numeric

Daca o configuratie de 8-regine are 5 perechi care se ataca:

$$f(s) = 28 - 5 = 23.$$

Daca alta are 4 perechi care se ataca:

$$f(s) = 28 - 4 = 24.$$

A doua configuratie este mai buna, deoarece are fitness mai mare.

13 Selectia

13.1 Scopul selectiei

Selectia decide:

- cine devine parinte;
- cine supravietuieste in generatia urmatoare.

Principiu:

fitness mare $\Rightarrow$ sanse mai mari de selectie.

13.2 Selectie proportionala

Pentru indivizi cu fitnessuri:

$$F_1, F_2, \dots, F_m,$$

probabilitatea de selectie este:

$$p_i = \frac{F_i}{F_1 + F_2 + \dots + F_m}.$$

13.3 Exemplu de examen

Avem fitnessurile:

$$24, \quad 23, \quad 20, \quad 11.$$

Suma este:

$$24 + 23 + 20 + 11 = 78.$$

Probabilitatile sunt:

$$p_1 = \frac{24}{78} \approx 0.31,$$

$$p_2 = \frac{23}{78} \approx 0.29,$$

$$p_3 = \frac{20}{78} \approx 0.26,$$

$$p_4 = \frac{11}{78} \approx 0.14.$$

Deci primul individ are aproximativ 31% sansa sa fie selectat.

13.4 Problema valorilor negative

Selectia proportionala cere fitnessuri pozitive. Daca fitnessurile sunt negative, trebuie transformate.

Exemplu:

$$F = (-3, 2, 7).$$

Adaugam 4 la toate:

$$F' = (1, 6, 11).$$

Acum toate sunt pozitive si putem calcula probabilitati.

13.5 Selectie pe baza rangurilor

Nu folosim valorile absolute, ci ordinea indivizilor. Cel mai slab primeste rang mic, cel mai bun rang mare.

Avantaj: evita situatia in care un individ foarte bun domina complet populatia.

13.6 Selectie de tip turneu

Pasii:

1. Alegem aleator k indivizi.
2. Ii comparam dupa fitness.
3. Castiga cel mai bun dintre ei.

Caz frecvent:

$$k = 2.$$

13.7 Selectie prin trunchiere

Alegem direct cei mai buni k indivizi. Este determinista si pune presiune puternica pe imbunatatire, dar reduce diversitatea.

14 Crossover / incrucisare

14.1 Scop

Crossover combina informatia genetica a doi parinti pentru a produce urmasi.

14.2 Crossover cu un punct

Parinti:

$$P_1 = 11110000,$$

$$P_2 = 00001111.$$

Alegem punctul de taiere dupa pozitia 4:

$$1111|0000,$$

$$0000|1111.$$

Urmasi:

$$O_1 = 11111111,$$

$$O_2 = 00000000.$$

14.3 Crossover cu doua puncte

Parinti:

$$P_1 = 11110000,$$

$$P_2 = 00001111.$$

Taiem intre pozitiile 3 si 6:

$$P_1 = 111|100|00,$$

$$P_2 = 000|011|11.$$

Schimbam segmentul din mijloc:

$$O_1 = 11101100,$$

$$O_2 = 00010011.$$

14.4 Crossover uniform

Pentru fiecare gena alegem aleator daca vine de la parintele 1 sau parintele 2.

Exemplu:

$$P_1 = 11110000, \quad P_2 = 00001111.$$

Masca:

$$M = 10101010.$$

Daca $M_i = 1$ luam gena din P_1 , altfel din P_2 :

$$O = 10100101.$$

Important pentru examen: Pentru codificari speciale, ca permutari pentru N-regine, nu orice crossover este valid. Daca urmasul nu mai este permutare, trebuie reparat sau folosita o incrucisare speciala.

15 Mutatia

15.1 Scop

Mutatia introduce variatie noua si ajuta algoritmul sa nu se blocheze in maxime/minime locale.

15.2 Mutatie binara

Pentru siruri binare, mutatia inseamna complementarea unei gene:

$$0 \leftrightarrow 1.$$

Exemplu:

$$101101 \rightarrow 101001,$$

daca gena de pe pozitia 4 s-a schimbat din 1 in 0.

15.3 Probabilitate de mutatie

Daca un cromozom are m gene, se foloseste adesea:

$$P_m = \frac{1}{m},$$

pentru ca, in medie, sa se modifice o gena pe cromozom.

15.4 Mutatie pentru permutari

Pentru N-regine, daca cromozomul este o permutare, putem folosi:

- swap mutation: schimbam doua pozitii intre ele;
- insertion mutation: mutam o valoare in alta pozitie;
- inversion mutation: inversam un segment.

Exemplu:

$$(3, 2, 7, 5, 4, 1, 6, 8)$$

Schimbam pozitiile 2 si 6:

$$(3, 1, 7, 5, 4, 2, 6, 8).$$

16 Exercițiu complet: algoritm genetic pentru 8-regine

16.1 Cerința

Propuneți un algoritm genetic pentru problema celor 8 regine: codificare, fitness, selecție, crossover, mutație.

Rezolvare. Codificare. Folosesc o permutare:

$$(s_1, s_2, \dots, s_8),$$

unde s_i este linia reginei din coloana i .

Fitness. Numarul maxim de perechi care nu se ataca este:

$$\binom{8}{2} = 28.$$

Dacă $a(s)$ este numărul de perechi care se ataca:

$$f(s) = 28 - a(s).$$

Soluția optimă are:

$$f(s) = 28.$$

Selecție. Pot folosi selecție proporțională:

$$p_i = \frac{f_i}{\sum_j f_j},$$

sau selecție de tip turneu.

Crossover. Deoarece folosesc permutări, aleg un crossover care păstrează permutarea sau repar urmașul după crossover.

Mutație. Folosesc swap mutation: aleg două poziții și le interschimb.

Oprire. Mă opresc când găsesc individ cu fitness 28 sau după un număr maxim de generații.

17 Exercițiu complet: algoritm genetic pentru rucsac

17.1 Cerința

Avem obiecte cu valori v_i și greutăți w_i , iar capacitatea rucsacului este C . Propuneți codificare și fitness.

Rezolvare. Codificare.

$$(s_1, s_2, \dots, s_n), \quad s_i \in \{0, 1\}.$$

$s_i = 1$ înseamnă că obiectul i este ales.

Valoarea totală.

$$V(s) = \sum_i s_i v_i.$$

Greutatea totală.

$$W(s) = \sum_i s_i w_i.$$

Fitness. Dacă $W(s) \leq C$:

$$f(s) = V(s).$$

Dacă $W(s) > C$, penalizăm:

$$f(s) = 0$$

sau:

$$f(s) = V(s) - \lambda(W(s) - C).$$

Crossover. Pot folosi crossover cu un punct sau două puncte.

Mutație. Schimb o gena: $0 \rightarrow 1$ sau $1 \rightarrow 0$.

18 Exercițiu complet: selecție proporțională

18.1 Cerința

O populație are 5 indivizi cu fitness:

$$F = (10, 20, 30, 25, 15).$$

Calculați probabilitățile de selecție.

Rezolvare. Suma fitnessurilor:

$$10 + 20 + 30 + 25 + 15 = 100.$$

Probabilități:

$$p_1 = 10/100 = 0.10,$$

$$p_2 = 20/100 = 0.20,$$

$$p_3 = 30/100 = 0.30,$$

$$p_4 = 25/100 = 0.25,$$

$$p_5 = 15/100 = 0.15.$$

Individul 3 are cea mai mare probabilitate de selecție, pentru că are fitnessul cel mai mare.

19 ExpectiMax - recapitulare utilă

Chiar dacă ai deja un PDF separat pentru căutări adversariale, Curs6 include și ExpectiMax, deci merita reținut pe scurt.

19.1 Când folosim ExpectiMax

Folosim ExpectiMax când rezultatul nu este ales de un adversar rațional MIN, ci de o incertitudine:

- zaruri;
- acțiuni care pot esua;
- adversar aleator;
- mediu imprevizibil.

19.2 Reguli

- Nod MAX: ia maximumul copiilor.
- Nod sansa/EXP: ia media ponderată a copiilor.

Formula:

$$V(n) = \sum_i p_i V_i.$$

Dacă nu se dau probabilități, folosim media aritmetică.

19.3 Exemplu

Un nod de sansă are trei rezultate:

$$8, \quad 24, \quad -12$$

cu probabilități:

$$\frac{1}{2}, \quad \frac{1}{3}, \quad \frac{1}{6}.$$

Atunci:

$$V = \frac{1}{2} \cdot 8 + \frac{1}{3} \cdot 24 + \frac{1}{6} \cdot (-12) = 4 + 8 - 2 = 10.$$

Important pentru examen: Alpha-Beta nu se aplică direct la ExpectiMax, deoarece un fiu nevizitat poate avea valoare foarte mare și poate schimba media.

20 Subiecte tip examen

Subiect 1 - PSC

Formulati Sudoku ca problema de satisfacere a constrangerilor.

Raspuns asteptat: variabilele sunt celulele, domeniile sunt $\{1, \dots, 9\}$, constrangerile sunt distinctie pe fiecare linie, coloana si patrat 3×3 .

Subiect 2 - Forward checking

Avem $A \neq B$, $A \neq C$, $B \neq C$, domenii $\{R, V, A\}$. Daca alegem $A = R$, ce se intampla?

Raspuns: eliminam R din domeniile lui B si C , deci $D(B) = D(C) = \{V, A\}$.

Subiect 3 - Hill climbing

Explicati de ce Hill Climbing nu este complet.

Raspuns: se poate bloca intr-un minim/maxim local, unde niciun vecin nu este mai bun, desi exista solutie mai buna in alta regiune a spatiului.

Subiect 4 - 8-regine

Pentru 8-regine, cati vecini are o stare daca mutam o regina pe alta linie in aceeasi coloana?

Raspuns: 8 coloane, fiecare regina poate fi mutata pe 7 alte linii, deci $8 \cdot 7 = 56$ vecini.

Subiect 5 - selectie proportionala

Fitnessuri: 24, 23, 20, 11. Care este probabilitatea de selectie a primului individ?

Raspuns:

$$\frac{24}{24 + 23 + 20 + 11} = \frac{24}{78} \approx 31\%.$$

Subiect 6 - algoritm genetic

Care este rolul mutatiei?

Raspuns: introduce diversitate, permite explorarea unor regiuni noi si ajuta la evitarea blocarii in optime locale.

21 Fisa rapida de memorat

PSC/CSP

- $PSC = (X, D, C)$.
- X = variabile, D = domenii, C = constrangeri.
- Stare in cautare = asignare partiala.
- Stare initiala = asignarea vida.
- Stare scop = asignare completa si consistenta.
- Forward checking = dupa o asignare, sterg valorile incompatibile din domeniile vecinilor.

Hill Climbing

- Nu pastreaza frontiera.
- Porneste dintr-o stare curenta.
- Alege cel mai bun vecin.
- Se opreste cand nu exista vecin mai bun.
- Incomplet si suboptimal.
- Probleme: maxim/minim local, platou, creasta.

Algoritmi genetici

- Populatie = mai multe solutii candidate.
- Cromozom = codificarea unei solutii.
- Fitness = calitatea solutiei.
- Selectie = exploatare.
- Crossover + mutatie = explorare.
- Probabilitate selectie proportionala: $p_i = F_i / \sum_j F_j$.
- Pentru N-regine: $f = 28 - \#conflicte$.

ExpectiMax

- MAX ia maximul.
- EXP ia media ponderata.
- Formula: $V = \sum_i p_i V_i$.
- Nu merge alpha-beta direct pe ExpectiMax.

Surse

Material sintetizat din Curs6-s23 si Curs7-s23: Alpha-Beta, ExpectiMax, probleme de satisfacere a constrangerilor, cautare locala, hill climbing, calcul evolutiv si algoritmi genetici.